

Scalability

Introduction

Scalability is the ability of a system to handle increasing load by efficiently adding resources without major redesign, downtime, or excessive cost.

It ensures that as users, traffic, data volume, or transactions grow, the system continues to perform reliably.

Formal Definition

Scalability is:

The ability of a system to adjust CPU, memory, storage, and network resources easily and cost-effectively to support increasing or decreasing load.

Why Scalability Matters

Without scalability:

- Website becomes slow
- Requests fail during traffic spikes
- Database overload occurs
- User experience degrades
- Revenue loss happens
- Downtime risk increases

Real World Examples

Netflix

Millions of concurrent users globally require scalable streaming infrastructure.

Amazon

During sales events, traffic can multiply instantly.

Instagram

Rapid user growth requires scaling storage, feeds, and media delivery.

Types of Scaling

1. Vertical Scaling (Scale Up)

Increase power of a single server.

Upgrade Examples:

- More CPU cores
- More RAM
- Faster SSD
- Better network card
- GPU acceleration

Example

Current server:

- 4 CPU

- 8 GB RAM

Upgrade to:

- 32 CPU
- 128 GB RAM

Advantages

- Easy to implement
- No major architecture changes
- Useful for small systems
- Fast short-term solution

Disadvantages

1. Hardware Limits

Single machine has max CPU/RAM/storage limits.

2. Expensive

Large enterprise servers cost disproportionately more.

3. Single Point of Failure

If machine crashes, system goes down.

4. Downtime Risk

Some upgrades require restart/migration.

2. Horizontal Scaling (Scale Out)

Add more servers and distribute traffic.

Instead of stronger machine → add many machines.

Example

1 server handling 1,000 req/sec

Scale to:

10 servers handling 10,000 req/sec

Advantages

- Near unlimited growth
- Better fault tolerance
- High availability
- Cost-effective commodity servers
- Easier global deployment

Disadvantages

- More architecture complexity
- Need load balancer
- Distributed systems problems
- Network latency
- Data consistency challenges

Interview Rule

When people say **scalable system**, they usually mean **horizontal scalability**.

Visual Comparison

None

Vertical Scaling:

App -> Bigger Server

Horizontal Scaling:

-> Server 1

Users -> Load Balancer -> Server 2

-> Server 3

-> Server N

What Resources Need Scaling?

Scalability means increasing one or more bottlenecks:

Compute

- CPU
- Memory

Storage

- SSD / HDD
- Object storage

Network

- Bandwidth
- CDN

Database

- Read replicas
- Sharding

Queue Capacity

- Kafka / RabbitMQ consumers

Questions to Ask in Interview

Always quantify scale.

Traffic Questions

- DAU / MAU?
- Peak concurrent users?

- Requests per second?
- Read-heavy or write-heavy?
- Burst traffic expected?

Data Questions

- Data generated per day?
- Image/video uploads?
- Retention period?
- Analytics volume?

Performance Questions

- Target latency?
- P99 requirement?
- SLA uptime?

Important Metrics

QPS / RPS

Queries or Requests Per Second.

Example:

10,000 requests/sec

Throughput

Amount processed per unit time.

Example:

1 million messages/hour

Concurrency

How many users connected simultaneously.

Traffic Spike

Sudden temporary load increase.

Example:

Flash sale, cricket final, festival sale.

Types of Scalability

1. Read Scalability

Handle increasing reads.

Solutions:

- Cache
- CDN
- Read replicas

- Search engine

2. Write Scalability

Handle increasing writes.

Solutions:

- Partitioning
- Queue buffering
- Batch writes
- Event streaming

3. Storage Scalability

Store growing data.

Solutions:

- Sharding
- Object storage
- Data archival

4. Geographic Scalability

Serve users globally.

Solutions:

- Multi-region deployment
- GeoDNS
- CDN edge locations

Techniques to Achieve Scalability

1. Load Balancer

Distributes traffic across servers.

Examples:

- NGINX
- HAProxy
- Amazon Web Services ALB

2. Stateless Services

Store session externally.

Any server can handle request.

3. Caching

Use:

- Redis
- Memcached

Reduces DB load.

4. Database Scaling

Read Replicas

For SELECT queries.

Sharding

Split data across DB nodes.

5. CDN

Serve static content globally.

Example:

- Cloudflare
- Amazon Web Services CloudFront

6. Async Processing

Use queues:

- Apache Kafka
- RabbitMQ

7. Auto Scaling

Automatically add/remove servers based on CPU/QPS.

Example Growth Journey

Stage 1

Single server

Stage 2

Separate app + DB

Stage 3

Load balancer + many app servers

Stage 4

Cache + read replicas

Stage 5

Sharding + queues + CDN + multi-region

Scalability vs Performance

Performance

How fast system is now.

Scalability

How well system performs when load increases.

A fast system may still fail at scale.

Scalability vs Availability

Scalability

Handles more load.

Availability

System stays online.

Need both.

Common Bottlenecks

- Database joins
- Slow disk I/O
- Memory exhaustion
- Lock contention
- Network saturation
- CPU spikes
- Monolith architecture

Tradeoffs

Improve Scalability	Cost
More servers	Higher infra cost
Sharding	Operational complexity
Caching	Invalidation problems

Replication	Consistency lag
Microservices	More debugging

Interview Answer Structure

When asked scalability:

Step 1

Estimate traffic.

Step 2

Find bottleneck.

Step 3

Scale stateless layer first.

Step 4

Add caching.

Step 5

Scale database.

Step 6

Global expansion if needed.

Final Summary

Scalability is the backbone of modern distributed systems.

A scalable architecture should:

- Grow with traffic
- Maintain low latency
- Stay cost-efficient
- Recover from spikes
- Avoid redesign every year